Révisions en vrac

Quentin Fortier

March 29, 2026

Représentations des nombres : Conversion en base 10

Théorème (admis)

Soit $b \in \mathbb{N}^*$. Tout entier $n \in \mathbb{N}^*$ peut s'écrire de façon unique sous la forme : $n = n_{p-1} \times b^{p-1} + \cdots + n_1 \times b + n_0$ où $0 \leq n_i < b$, $\forall i$.

Question

Écrire une fonction `to_base10` telle que `to_base10 b l` renvoie le nombre dont la représentation en base `b` est `l`.

Représentations des nombres : Conversion en base 10

```
let rec to_base10 b l = match l with
    | [] -> 0
    | e::q -> e + b*(to_base10 b q)
```

Représentations des nombres : Conversion depuis la base 10

On veut maintenant passer de la base décimale à une autre base.

Pour passer un nombre n de la base 10 à la base 2, il faut trouver les n_i tels que:

$$n = n_{p-1} \times 2^{p-1} + ... + n_1 \times 2 + n_0$$

Représentations des nombres : Conversion depuis la base 10

On veut maintenant passer de la base décimale à une autre base.

Pour passer un nombre n de la base 10 à la base 2, il faut trouver les n_i tels que:

$$n = n_{p-1} \times 2^{p-1} + ... + n_1 \times 2 + n_0$$

$$\Leftrightarrow n = \underbrace{2}_{b} \times \underbrace{(n_{p-1} \times 2^{p-2} + ... + n_1)}_{q} + \underbrace{n_0}_{r}$$

Représentations des nombres : Conversion depuis la base 10

On veut maintenant passer de la base décimale à une autre base.

Pour passer un nombre n de la base 10 à la base 2, il faut trouver les n_i tels que:

$$n = n_{p-1} \times 2^{p-1} + ... + n_1 \times 2 + n_0$$

$$\Leftrightarrow n = \underbrace{2}_{b} \times \underbrace{(n_{p-1} \times 2^{p-2} + ... + n_1)}_{q} + \underbrace{n_0}_{r}$$

Ainsi, n_0 est le reste de la division de n par 2, n_1 est le reste de la division de q par 2, et ainsi de suite...

Représentations des nombres : Conversion depuis la base 10

Passons 98 de la base 10 à la base 2 :

$$
\begin{array}{r|l}
98 & 2 \\ \cline{2-2}
\mathbf{0} & 49 \\ \cline{2-2}
\mathbf{1} & 24 \\ \cline{2-2}
\mathbf{0} & 12 \\ \cline{2-2}
\mathbf{0} & 6 \\ \cline{2-2}
\mathbf{0} & 3 \\ \cline{2-2}
\mathbf{1} & 1 \\ \cline{2-2}
\mathbf{1} & 0
\end{array}
\qquad
\begin{array}{l}
2 \\ \hline
2 \\ \hline
2 \\ \hline
2 \\ \hline
2 \\ \hline
2 \\ \hline
2
\end{array}
$$

Représentations des nombres : Conversion depuis la base 10

Passons 98 de la base 10 à la base 2 :

$$
\begin{array}{r|l}
98 & 2 \\ \cline{2-2}
\mathbf{0} & 49 \\ \cline{2-2}
\mathbf{1} & 24 \\ \cline{2-2}
\mathbf{0} & 12 \\ \cline{2-2}
\mathbf{0} & 6 \\ \cline{2-2}
\mathbf{0} & 3 \\ \cline{2-2}
\mathbf{1} & 1 \\ \cline{2-2}
\mathbf{1} & 0
\end{array}
\qquad
\begin{array}{l}
2 \\ \hline
2 \\ \hline
2 \\ \hline
2 \\ \hline
2 \\ \hline
2 \\ \hline
2
\end{array}
$$

Donc $\boxed{98 = 1100010_2}$ (on lit les restes à l'envers).

Représentations des nombres : Conversion depuis la base 10

Exercice

Écrire une fonction `from_base10` prenant une base `b` et un nombre `n` et renvoyant la liste des chiffres de `n` en base `b`.

Représentations des nombres : Conversion depuis la base 10

Exercice

Écrire une fonction `from_base10` prenant une base `b` et un nombre `n` et renvoyant la liste des chiffres de `n` en base `b`.

```
let rec from_base10 b n =
    if n = 0 then []
    else (n mod b)::from_base10 b (n / b)
```

Invariant

Définition

Un invariant est une propriété qui doit être vérifiée à tout moment pendant l'exécution d'un programme et qui permet de garantir la correction du programme.

Invariant

Exemple d'invariant :

```
let tri t =
    let n = Array.length t in
    for i = 0 to n - 1 do
        for j = i + 1 to n - 1 do
            if t.(i) > t.(j) then
                let tmp = t.(i) in
                t.(i) <- t.(j);
                t.(j) <- tmp
        done
```

Invariant

Exemple d'invariant :

```
let tri t =
    let n = Array.length t in
    for i = 0 to n - 1 do
        for j = i + 1 to n - 1 do
            if t.(i) > t.(j) then
                let tmp = t.(i) in
                t.(i) <- t.(j);
                t.(j) <- tmp
        done
```

« à la ième itération de la boucle extérieure, `t.(i)` est le ième élément le plus petit de `t` » ne convient pas.

Invariant

Exemple d'invariant :

```
let tri t =
    let n = Array.length t in
    for i = 0 to n - 1 do
        for j = i + 1 to n - 1 do
            if t.(i) > t.(j) then
                let tmp = t.(i) in
                t.(i) <- t.(j);
                t.(j) <- tmp
        done
```

« à la ième itération de la boucle extérieure, `t.(i)` est le ième élément le plus petit de `t` » ne convient pas.
Plutôt : « à la ième itération de la boucle extérieure, les i premiers éléments de `t` sont un triés des i éléments les plus petits de `t` ».

Invariant

Un invariant comme commentaire peut aussi aider à comprendre le code :

```
bool dichotomie(int e, int* t, int n) {
    // t est un tableau trié
    int i = 0, j = n - 1;
    while (i <= j) {
        // si e est dans t alors e est dans t[i..j]
        int m = (i + j) / 2;
        if (t[m] == e)
            return 1;
        if (t[m] <= e) i = m + 1;
        else j = m - 1;
    }
    return 1;
}
```

Tests

Malheureusement, il est difficile de trouver des invariants.
À la place, on peut utiliser des tests, qui consistent à vérifier, sur des entrées particulières, que la sortie du programme est conforme à la spécification.

Tests

Malheureusement, il est difficile de trouver des invariants.
À la place, on peut utiliser des tests, qui consistent à vérifier, sur des entrées particulières, que la sortie du programme est conforme à la spécification.

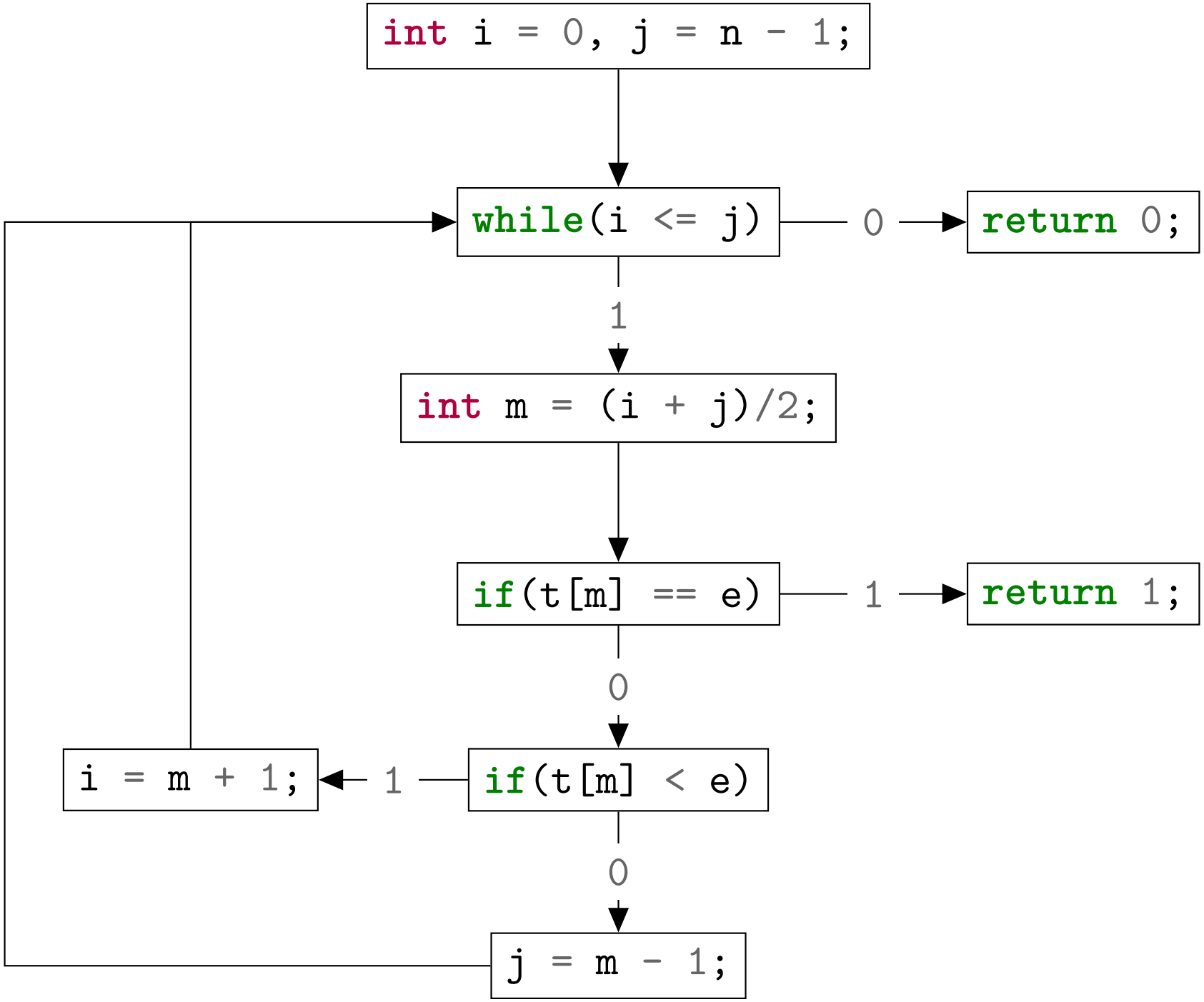
Types de tests :

- Test unitaire : Une petite portion de code à la fois (une fonction, par exemple).
- Test fonctionnel : Conformité du logiciel développé à la spécification.
- Test de non-régression : Teste que l'ajout de code n'a pas introduit d'erreurs sur le code déjà existant.
- Test de performance : Rapidité, précision...
- ...

Tests : Graphe de flot de contrôle (CFG)

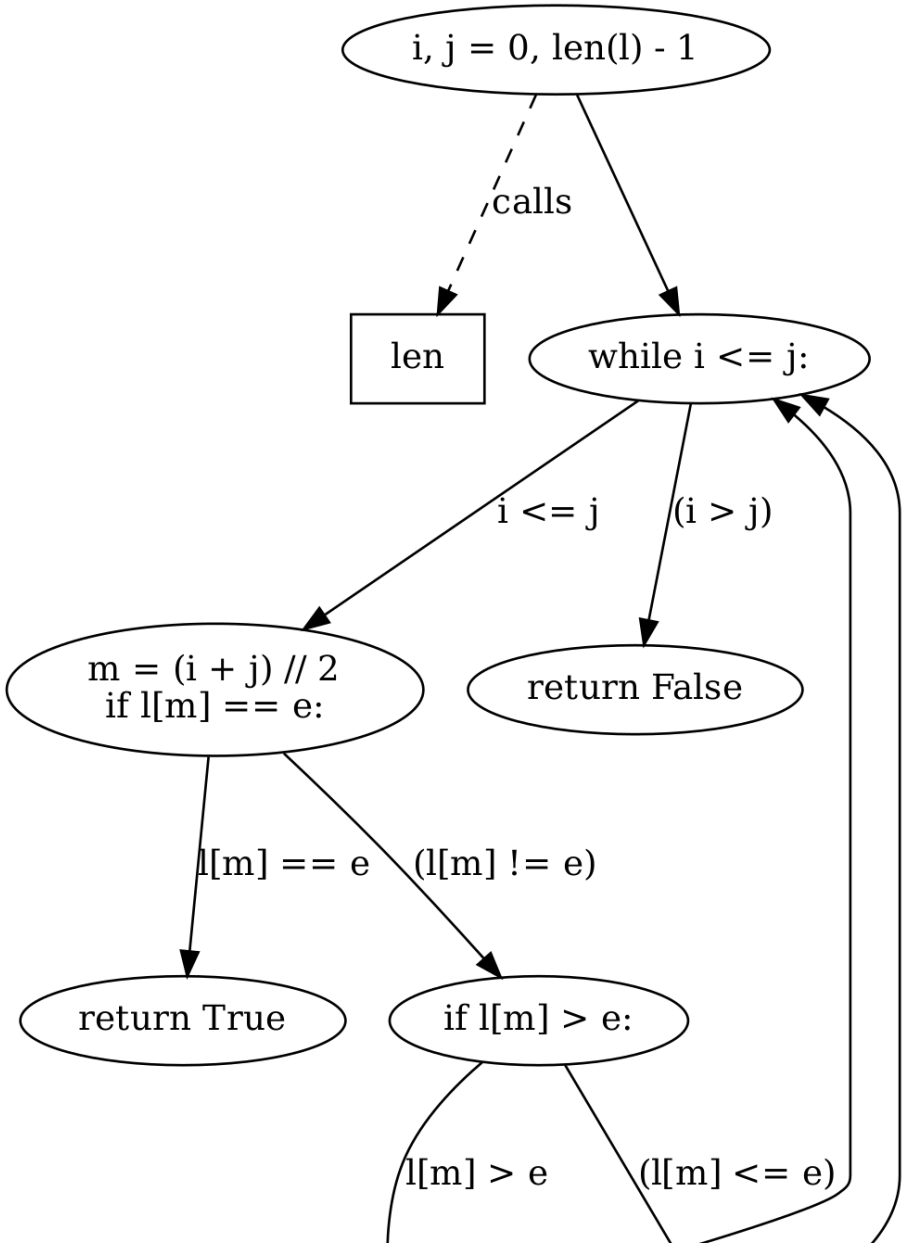
Le graphe de flot de contrôle (Control Graph Flow) d'un programme est un graphe orienté avec :

- (Un sommet d'entrée, un sommet de sortie.)
- Un sommet pour chaque bloc d'instructions consécutives.
- Un sommet pour chaque condition ou boucle.
- Des arcs pour relier des blocs consécutifs. Les arcs sortants d'un condition ou une boucle sont étiquetées par la condition.


int i = 0, j = n - 1;
while(i <= j)
0
return 0;
1
int m = (i + j)/2;
if(t[m] == e)
1
return 1;
0
i = m + 1;
1
if(t[m] < e)
0
j = m - 1;

Tests : Graphe de flot de contrôle (CFG)

Génération automatique de CFG en Python avec staticfg :



Tests : Graphe de flot de contrôle (CFG)

Une exécution du programme (sur une entrée donnée) correspond à un chemin dans le CFG.

Tests : Graphe de flot de contrôle (CFG)

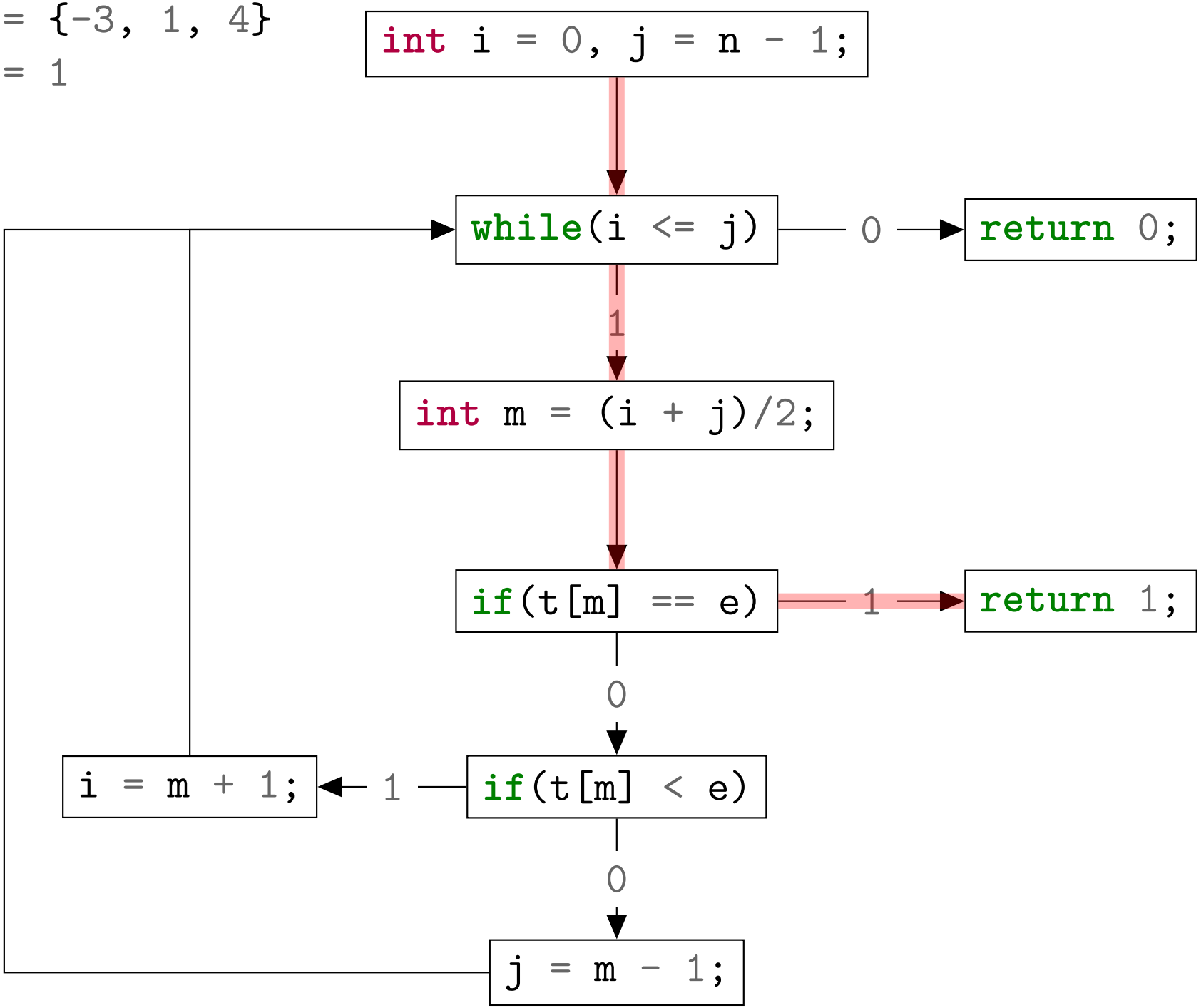
Une exécution du programme (sur une entrée donnée) correspond à un chemin dans le CFG.

Un chemin est faisable s'il correspond à l'exécution d'un programme sur une entrée donnée.

Exercice

Donner un exemple de CFG avec un chemin infaisable.

t = {-3, 1, 4}
e = 1


int i = 0, j = n - 1;
while(i <= j)
0
return 0;
1
int m = (i + j)/2;
if(t[m] == e)
1
return 1;
0
i = m + 1;
1
if(t[m] < e)
0
j = m - 1;

Tests : Couverture de code

Pour tester en utilisant le CFG, on peut choisir des tests qui couvrent un certain ensemble de chemins :

- Couverture de tous les chemins (*path cover*) : On choisit autant de tests qu'il y a de chemins possible.
  Mais il peut y avoir une infinité de chemins...

Tests : Couverture de code

Pour tester en utilisant le CFG, on peut choisir des tests qui couvrent un certain ensemble de chemins :

- Couverture de tous les chemins (*path cover*) : On choisit autant de tests qu'il y a de chemins possible.
  Mais il peut y avoir une infinité de chemins...
- Couverture de toutes les instructions (*statement cover*) : On écrit des tests de façon à ce que chaque instruction soit exécuté au moins une fois.

Tests : Couverture de code

Pour tester en utilisant le CFG, on peut choisir des tests qui couvrent un certain ensemble de chemins :

- Couverture de tous les chemins (*path cover*) : On choisit autant de tests qu'il y a de chemins possible.
  Mais il peut y avoir une infinité de chemins...
- Couverture de toutes les instructions (*statement cover*) : On écrit des tests de façon à ce que chaque instruction soit exécuté au moins une fois.
- Couverture de toutes les branches (*branch cover*) : On écrit des tests de façon à ce que par chaque arc passe au moins un chemin.

Tests : Couverture de code

Pour tester en utilisant le CFG, on peut choisir des tests qui couvrent un certain ensemble de chemins :

- Couverture de tous les chemins (*path cover*) : On choisit autant de tests qu'il y a de chemins possible.
  Mais il peut y avoir une infinité de chemins...
- Couverture de toutes les instructions (*statement cover*) : On écrit des tests de façon à ce que chaque instruction soit exécuté au moins une fois.
- Couverture de toutes les branches (*branch cover*) : On écrit des tests de façon à ce que par chaque arc passe au moins un chemin.

Taux de couverture : $\dfrac{\text{nombre sommets/arcs parcourus}}{\text{nombre sommets/arcs}}$

Exceptions

On peut définir un nouveau constructeur d'exception en OCaml, de type `exn` :

```
exception E (* exception sans argument *)
exception E2 of string (* exception avec un argument de type strin
exception E3 of int * string (* exception avec un argument de type
```

On peut aussi utiliser les exceptions prédéfinies : `Failure`, `Not_found`, `Division_by_zero`…

Exceptions

On peut lever une exception avec `raise : exn -> 'a` et la récupérer avec `try ... with ...` :

```
try raise (E2 "erreur") with
    | E -> ...
    | E2 s -> ... (* exécuté avec s = "erreur" *)
    | E3 (i, s) -> ...
```

```
let inverse x =
    try 1 / x with
    | Division_by_zero -> max_int
```

Exercice

Expliquer pourquoi `(raise E) (raise E)` est accepté par l'interpréteur OCaml.

Exceptions

Exemple : lecture d'un fichier. L'exception `End_of_file` est levée par `input_line` lorsque la fin du fichier est atteinte.

```
let read_lines filename =
    let f = open_in filename in
    let rec aux () =
        try
            let line = input_line f in
            line :: aux ()
        with
        | End_of_file ->
            close_in f;
            []
    in
    aux ()
```

Triplet de Hoare (HP)

Définition

Si P est une précondition (booléen), Q est une postcondition et c un programme, le triplet de Hoare $\{$`P`$\}$ c $\{$`Q`$\}$ signifie :
« Si la commande c, démarrée dans un etat initial satisfaisant P, termine, alors l'etat final satisfait Q »

Exemple : $\{$`x = 3`$\}$ `x := x + 1` $\{$`x = 4`$\}$.

Triplet de Hoare (HP)

Règles de déduction de la logique de Hoare :

$$\{\, P \,\}\ \texttt{skip}\ \{\, P \,\} \qquad\qquad \{\, Q[x \leftarrow a] \,\}\ x := a\ \{\, Q \,\}$$

$$\frac{\{\, P \,\}\ c_1\ \{\, Q \,\} \qquad \{\, Q \,\}\ c_2\ \{\, R \,\}}{\{\, P \,\}\ c_1; c_2\ \{\, R \,\}}$$

$$\frac{\{\, P \wedge b \,\}\ c_1\ \{\, Q \,\} \qquad \{\, P \wedge \neg b \,\}\ c_2\ \{\, Q \,\}}{\{\, P \,\}\ \texttt{if}\ b\ \texttt{then}\ c_1\ \texttt{else}\ c_2\ \{\, Q \,\}}$$

$$\frac{\{\, P \wedge b \,\}\ c\ \{\, P \,\}}{\{\, P \,\}\ \texttt{while}\ b\ \texttt{do}\ c\ \{\, P \wedge \neg b \,\}}$$

Triplet de Hoare (HP)

Why3 est un logiciel de verification de programmes, basé sur la logique de Hoare et avec un langage proche d'OCaml :

Triplet de Hoare (HP)

Why3 est un logiciel de verification de programmes, basé sur la logique de Hoare et avec un langage proche d'OCaml :

```
module SumGauss
  use int.Int
  use ref.Ref

  let sum_gauss (n: int) : int
    requires { n > 0 }
    returns  { r -> 2 * r = n * (n-1) }
  = let sum = ref 0 in
    for i = 1 to n - 1 do
      invariant { 2 * !sum = i * (i - 1)}
      sum := !sum + i
    done;
    !sum
end
```

Trouver les erreurs

Exercice

- Trouver les erreurs dans les programmes suivants.
- Les corriger.

Liste

```
let somme l =
    let s = 0 in
    for i = 0 to List.length l - 1 do
        s := s + l.(i)
    done;
    s
```

Liste

```
let somme l =
    let s = 0 in
    for i = 0 to List.length l - 1 do
        s := s + l.(i)
    done;
    s
```

- `t.(i)` existe seulement pour un tableau, pas une liste.
- Impossible de modifier une variable en OCaml à moins que ce soit une référence.

Liste

On distingue les structures de données :

- Persistantes (immuables, non modifiables) : listes, arbres...
- Mutables (modifiables) : tableaux, références...

Liste

```
let somme l =
    let s = 0 in
    for i = 0 to List.length l - 1 do
        s := s + l.(i)
    done;
    s
```

```
let rec somme l = match l with
    | [] -> 0
    | t::q -> t + somme q
```

Liste

```
let range n =
    let l = [] in
    for i = 0 to n - 1 do
        i::l
    done;
    l
```

Liste

```
let range n =
    let l = [] in
    for i = 0 to n - 1 do
        i::l
    done;
    l
```

- `i::l` ne modifie pas `l`, mais renvoie une nouvelle liste.
- Il n'y a pas de `append` en OCaml.
- Une « liste » en Python est en fait un tableau.

Liste

```
let range n =
    let l = [] in
    for i = 0 to n - 1 do
        i::l
    done;
    l
```

```
let rec range n =
    if n = -1 then []
    else (n - 1)::(range (n - 1))
```

return

```
let appartient t e =
    for i = 0 to Array.length t - 1 do
        if t.(i) = e then true
    done;
    false
```

return

```
let appartient t e =
    for i = 0 to Array.length t - 1 do
        if t.(i) = e then true
    done;
    false
```

Pas de `return` en OCaml.

return

```
let appartient t e =
    for i = 0 to Array.length t - 1 do
        if t.(i) = e then true
    done;
    false
```

```
let appartient t e =
    let r = ref false in
    for i = 0 to Array.length t - 1 do
        if t.(i) = e then r := true
    done;
    !r
```

match

```
let appartient l e = match l with
    | [] -> false
    | e::q -> true
    | t::q -> appartient q e
```

match

```
let appartient l e = match l with
    | [] -> false
    | e::q -> true
    | t::q -> appartient q e
```

e est une nouvelle variable dans le **match**, qui écrase le e en argument.

match

```
let appartient l e = match l with
    | [] -> false
    | e::q -> true
    | t::q -> appartient q e
```

```
let appartient e l = match l with
    | [] -> false
    | t::q -> if t = e then true
        else appartient e q
```

Passage d'arguments

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

Passage d'arguments

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

Un appel `swap(x, y)` se fait par copie des arguments : `a` et `b` sont des copies de `x` et `y` (qui ne sont pas modifiées).

Passage d'arguments

Passage d'argument par valeur/copie

```
void f(int x) {
    x = 42;
}

int y = 0;
f(y); // y n'est pas modifié
```

```
def f(x):
    x = 42

y = 0
f(y) # y n'est pas modifié
```

Passage d'arguments

Passage d'argument par valeur/copie

```
void f(int x) {
    x = 42;
}

int y = 0;
f(y); // y n'est pas modifié
```

```
def f(x):
    x = 42

y = 0
f(y) # y n'est pas modifié
```

Passage d'argument par adresse/référence

```
void f(int* x) {
    *x = 42;
}

int y = 0;
f(&y); // y est modifié
```

```
let f t =
    t.(0) <- 42

let y = [|0|] in
f y (* y est modifié *)
```

Passage d'arguments

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

Passage d'arguments

```
void swap(int a, int b) {
    int tmp = a;
    a = b;
    b = tmp;
}
```

```
void swap(int* a, int* b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

Variable locale

```
float* zero(){
    float t[2];
    t[0] = t[1] = 0;
    return t;
}
```

Variable locale

```
float* zero(){
    float t[2];
    t[0] = t[1] = 0;
    return t;
}
```

```
float* zero(){
    float* t = {0, 0};
    return t;
}
```

Variable locale

```
float* zero(){
    float t[2];
    t[0] = t[1] = 0;
    return t;
}
```

```
float* zero(){
    float* t = {0, 0};
    return t;
}
```

t est une variable locale qui est détruite à la fin de la fonction.

Variable locale

```
float* zero(){
    float* t = {0, 0};
    return t;
}
```

```
float* zero(){
    float* t = (float*)malloc(2 * sizeof(float));
    t[0] = t[1] = 0;
    return t;
}
```

Variable locale

```
typedef struct {
    int* m;
    int n;
} graphe;

graphe g;
```

Exercice

Comment accéder à `m` dans `g` ?

Variable locale

```c
typedef struct {
    int* m;
    int n;
} graphe;

void f(graphe g) {
    g.m[0] = 42;
}

int main() {
    graphe g;
    g.m = (int*)malloc(sizeof(int));
    g.m[0] = 0;
    f(g);
    printf("%d\n", g.m[0]);
}
```

Exercice

Qu'affiche le `printf` ?

Modèle entité-association en SQL

Entité

Une entité est un ensembles d'objets similaires que l'on souhaite stocker.

Exemple : Livre, auteur...

Modèle entité-association en SQL

Entité

Une entité est un ensembles d'objets similaires que l'on souhaite stocker.

Exemple : Livre, auteur...

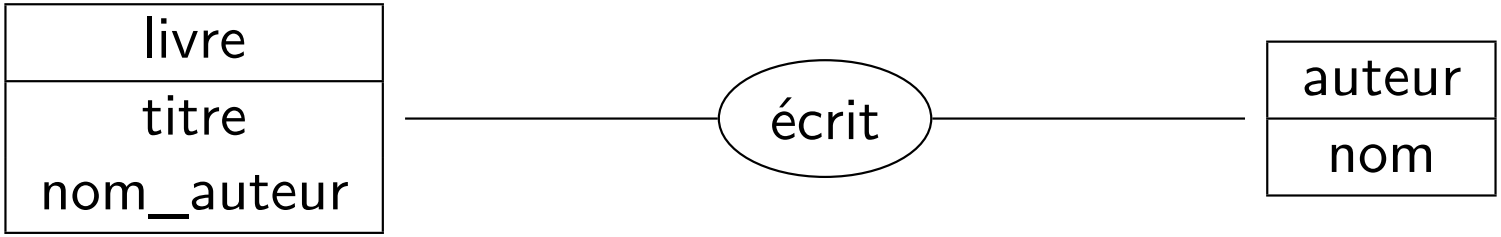
Définition

Une association (ou : relation) est une relation entre plusieurs entités.
Une association est binaire si elle met en relation deux entités.

Exemple : Un auteur *écrit* un livre.

Modèle entité-association en SQL : Représentation

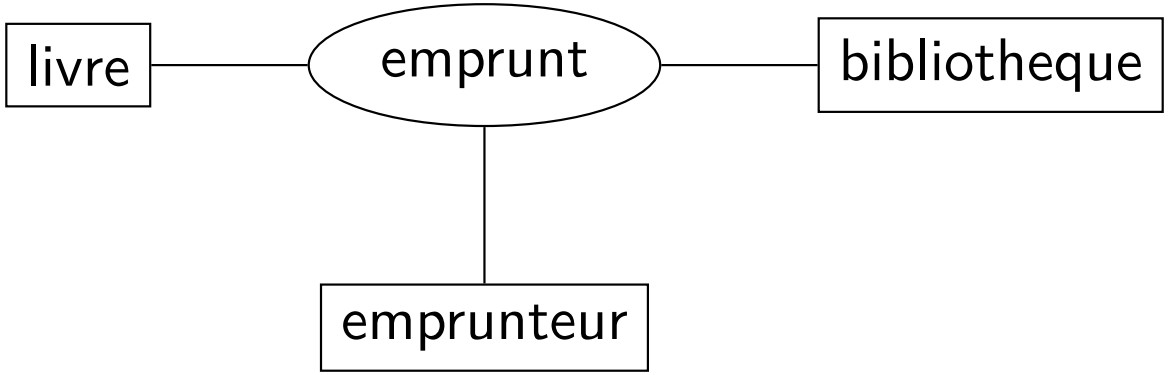
Représentation sous forme de diagramme :



Modèle entité-association en SQL : Représentation

Une relation n-aire peut être transformée en relation binaire en introduisant une nouvelle entité pour la relation.

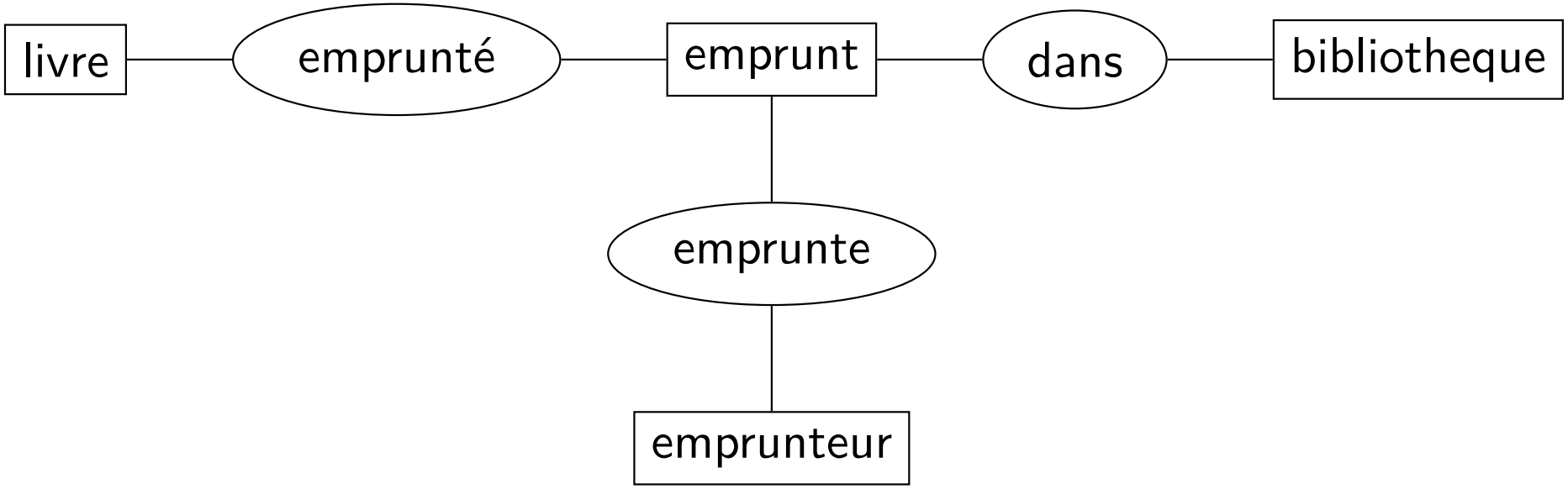
Exemple :



Modèle entité-association en SQL : Représentation

Une relation n-aire peut être transformée en relation binaire en introduisant une nouvelle entité pour la relation.

Exemple :



Modèle entité-association en SQL : Cardinalité

Définition

On peut spécifier le lien entre une entité et une association avec un couple (n, p) indiquant le nombre minimum et maximum de fois que l'entité peut apparaître dans l'association ($p = *$ s'il n'y a pas de maximum).

Modèle entité-association en SQL : Cardinalité

Définition

On peut spécifier le lien entre une entité et une association avec un couple (n, p) indiquant le nombre minimum et maximum de fois que l'entité peut apparaître dans l'association ($p = *$ s'il n'y a pas de maximum).

Exemples :

- Un livre a été écrit par au moins une personne, sans limite supérieure. D'où la cardinalité $(1, *)$ pour le lien entre l'entité livre et l'association « écrit ».

Modèle entité-association en SQL : Cardinalité

Définition

On peut spécifier le lien entre une entité et une association avec un couple (n, p) indiquant le nombre minimum et maximum de fois que l'entité peut apparaître dans l'association ($p = *$ s'il n'y a pas de maximum).

Exemples :

- Un livre a été écrit par au moins une personne, sans limite supérieure. D'où la cardinalité $(1, *)$ pour le lien entre l'entité livre et l'association « écrit ».
- Une personne peut avoir écrit un nombre quelconque de livre. D'où la cardinalité $(0, *)$.

Modèle entité-association en SQL : Cardinalité

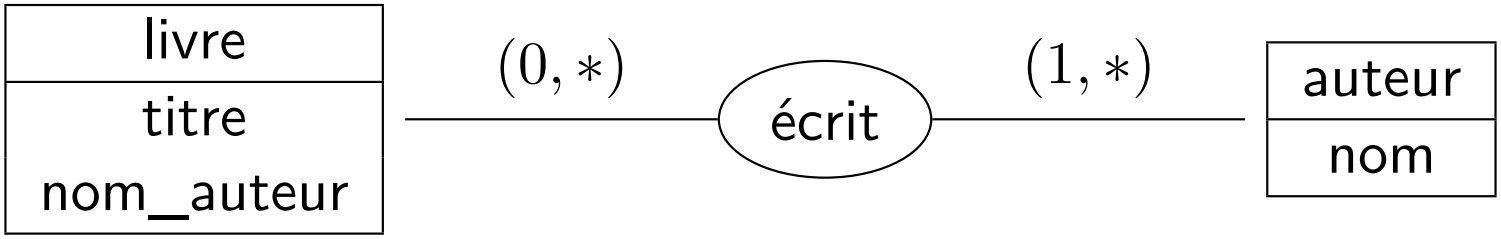
Définition

On peut spécifier le lien entre une entité et une association avec un couple (n, p) indiquant le nombre minimum et maximum de fois que l'entité peut apparaître dans l'association ($p = *$ s'il n'y a pas de maximum).

Exemples :

- Un livre a été écrit par au moins une personne, sans limite supérieure. D'où la cardinalité $(1, *)$ pour le lien entre l'entité livre et l'association « écrit ».
- Une personne peut avoir écrit un nombre quelconque de livre. D'où la cardinalité $(0, *)$.
- Si on suppose qu'une personne peut emprunter au plus 5 livres, alors le lien entre l'entité personne et l'association « emprunt » est de cardinalité $(0, 5)$.

Modèle entité-association en SQL : Cardinalité



Modèle entité-association en SQL : Cardinalité

Types possibles d'association entre deux entités :

- $1-1$ (*one-to-one*) : La borne supérieure vaut 1 pour les 2 entités. Exemple : L'association « dirige » est de type $1-1$ pour directeur_bibliotheque et bibliotheque.

Modèle entité-association en SQL : Cardinalité

Types possibles d'association entre deux entités :

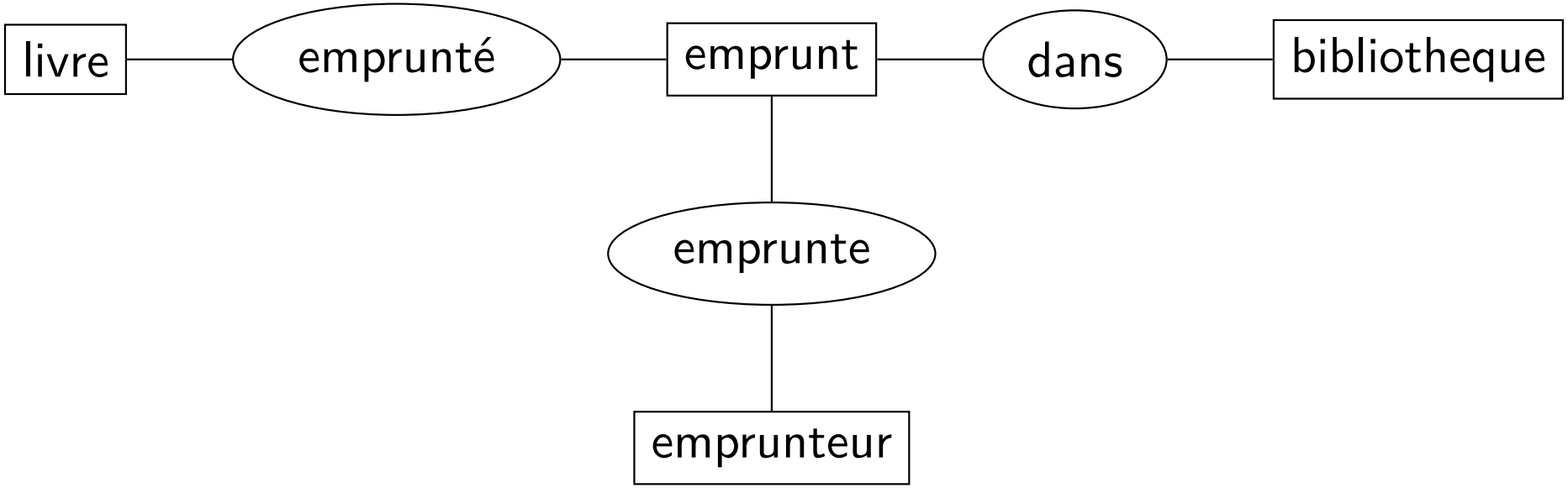
- $1-1$ (*one-to-one*) : La borne supérieure vaut 1 pour les 2 entités.
  Exemple : L'association « dirige » est de type $1-1$ pour directeur_bibliotheque et bibliotheque.
- $1-*$ (*one-to-many*) : La borne supérieure vaut 1 pour une entité et $*$ pour l'autre.
  Exemple : Chaque livre est écrit par un unique auteur, mais chaque auteur a pu écrire plusieurs livres.

Modèle entité-association en SQL : Cardinalité

Types possibles d'association entre deux entités :

- $1-1$ (*one-to-one*) : La borne supérieure vaut 1 pour les 2 entités.
  Exemple : L'association « dirige » est de type $1-1$ pour directeur_bibliotheque et bibliotheque.
- $1-*$ (*one-to-many*) : La borne supérieure vaut 1 pour une entité et $*$ pour l'autre.
  Exemple : Chaque livre est écrit par un unique auteur, mais chaque auteur a pu écrire plusieurs livres.
- $*-*$ (*many-to-many*) : La borne supérieure vaut $*$ des deux côtés.
  Exemple : L'association « est de type » entre la table des pokémons et des types est de type $*-*$ (à chaque pokémon peut correspondre plusieurs types et plusieurs pokémons peuvent avoir le même type).

Modèle entité-association en SQL : Cardinalité



Modèle entité-association en SQL : Conception de base de donnée

Pour concevoir une base de donnée :

- Utiliser une table par entité.

Modèle entité-association en SQL : Conception de base de donnée

Pour concevoir une base de donnée :

- Utiliser une table par entité.
- Pour chaque association entre a et b :
  - Si association $1-1$: Fusionner les tables a et b.

Modèle entité-association en SQL : Conception de base de donnée

Pour concevoir une base de donnée :

- Utiliser une table par entité.
- Pour chaque association entre a et b :
  - Si association $1-1$: Fusionner les tables a et b.
  - Si association $1-*$: Ajouter un attribut (clé étrangère) à b faisant référence à un a.

Modèle entité-association en SQL : Conception de base de donnée

Pour concevoir une base de donnée :

- Utiliser une table par entité.
- Pour chaque association entre a et b :
  - Si association $1-1$: Fusionner les tables a et b.
  - Si association $1-*$: Ajouter un attribut (clé étrangère) à b faisant référence à un a.
  - Si association $*-*$: Ajouter une table ayant 2 clé étrangère pour faire référence à a et b.